ENGINEERING & TECHNICAL PROJECT REPORT

SMART NETWORK PACKET SNIFFER v2

Real-Time Traffic Analysis, Multi-Layer Protocol Dissection, Heuristic Anomaly Detection & Telemetry Reporting System

A high-performance, modular Python network inspection framework designed for wire-speed packet capture, deep protocol decapsulation across Layers 2 through 7, real-time statistical bandwidth calculation, rule-based security anomaly alerting, and resilient session auditing on modern Windows architectures.

PROJECT DOMAIN
Computer Networks & Cybersecurity

CORE IMPLEMENTATION
Python 3.12, Scapy 2.7.0, Npcap Engine

OPERATING PLATFORM
Microsoft Windows (x64 Architecture)

DOCUMENT SPECIFICATION
v2.1 Comprehensive Architecture Report

SYSTEM MODULES
Capture, Dissection, Analytics, Logger, Report, CLI

TELEMETRY OUTPUTS
Structured CSV Logs & Archival TXT Summaries

Smart Packet Sniffer Development Team
October 2026
Confidential & Academic Project Material

00 Executive Summary & Abstract

Network packet sniffers represent fundamental instruments within modern computer systems, providing indispensable telemetry for performance diagnosis, security auditing, protocol validation, and traffic engineering. Traditional tools such as Wireshark or commercial Deep Packet Inspection (DPI) platforms often suffer from heavyweight memory footprints, excessive GUI latency, high runtime complexity, or closed architectures that prevent automated scripting and streamlined incident response.

Smart Network Packet Sniffer v2 is engineered to bridge the divide between low-level driver-based packet interception and automated application-layer telemetry. Built atop Python 3 and the Scapy packet manipulation suite, the system interfaces directly with the native Windows Npcap kernel driver to capture live promiscuous Ethernet, IP, and ARP traffic without dropped frames under moderate to high network loads.

Key Technical Milestones Achieved

- **Asynchronous GIL-Responsive Micro-Batching:** Implements a 0.5-second polling window that resolves the notorious Scapy Windows GIL-locking issue, enabling instantaneous `Ctrl+C` signal termination even on completely idle network adapters.
- **Full-Stack Multi-Layer Protocol Dissection:** Safely parses raw frames across Layer 2 (Ethernet, ARP), Layer 3 (IPv4, IPv6), Layer 4 (TCP with complete bitmask flag decoding, UDP, ICMP with RFC-compliant code mappings), and Layer 7 (well-known service heuristics).
- **Stateful Real-Time Analytics Engine:** Continuously measures throughput (Packets/sec, Bytes/sec, Kilobits/sec), payload distribution metrics, and endpoint talker rankings (source and destination IPs).
- **Rule-Based Security Anomaly Detection:** Evaluates traffic heuristics to flag TCP SYN flooding attempts, horizontal port scans (distinct port sweeping per host), and cleartext credentials transmitted over unencrypted legacy protocols (HTTP, Telnet, FTP).
- **Resilient Concurrent CSV Logging:** Employs auto-recovering file I/O with automatic lock detection and alternative filename redirection when target log files are opened in external applications such as Microsoft Excel.

> **Operational Summary**
> The system combines interactive terminal monitoring with dual-tier telemetry persistence. In empirical testing, the engine successfully processed over 2,500 frames without memory degradation while autonomously generating forensic audit trails.

00 Table of Contents & Specifications

Chapter 1: Introduction, Problem Statement & Objectives ..... Page 4

Chapter 2: System Architecture & Modular Pipeline Design ..... Page 5

Chapter 3: Packet Dissection & Multi-Layer Protocol Handling ..... Page 6

Chapter 4: Real-Time Analytics & Threat Anomaly Detection ..... Page 7

Chapter 5: Concurrency, Fault Tolerance & Logging Resilience ..... Page 8

Chapter 6: Empirical Verification, Benchmarking & Test Results ..... Page 9

Chapter 7: Security Privileges, Ethical Boundaries & Deployment ..... Page 10

Chapter 8: Conclusion, Future Roadmap & References ..... Page 11

System Environment & Hardware Specifications

Component / Parameter	Specification / Version	Operational Role in System
Runtime Interpreter	Python 3.12.x (64-bit)	Execution environment, standard library collections, and CLI runtime.
Core Packet Engine	Scapy 2.7.0	Network frame decapsulation, BPF filter parsing, and raw socket bindings.
Kernel Capture Driver	Npcap 1.7x+ (WinPcap API Mode)	Low-level kernel NDIS filter driver capturing raw frames off NIC buffers.
Operating System	Microsoft Windows 11 / 10 (x64)	Host OS; requires administrative privilege escalation for raw socket access.
Network Adapters	802.11ac Wi-Fi / Gigabit Ethernet	Physical hardware interfaces monitored in promiscuous capture mode.
Storage System	NTFS / Local SSD	Append-only CSV log repository (`logs/`) and report storage (`reports/`).

01 Introduction & Theoretical Background

1.1 Problem Statement & Motivations

In contemporary computer networks, raw packet data provides the absolute ground truth for all system communications. Diagnosing intermittent network bottlenecks, verifying firewall policies, and investigating security breaches all depend upon packet-level visibility. However, existing packet inspection tools present distinct operational challenges:

- **Excessive Resource Demands:** GUI-based sniffers like Wireshark carry substantial memory footprints that complicate remote headless deployment and rapid CLI triage.
- **Information Overload:** Unprocessed capture outputs inundate analysts with hexadecimal payloads without summarizing endpoint conversations or immediate security threats.
- **Windows Platform Concurrency Deadlocks:** On Windows platforms, Python-based sniffers frequently deadlock the Global Interpreter Lock (GIL) inside native C drivers, preventing clean termination via `Ctrl+C`.
- **File Sharing Violations:** Opening active CSV telemetry logs in analytical suites like Microsoft Excel triggers exclusive OS file locks, causing unhandled exceptions in standard sniffers.

1.2 System Objectives

The primary objective of **Smart Network Packet Sniffer v2** is to deliver a robust, modular, and resilient packet inspection engine tailored for Windows environments, featuring:

1. **Automated Adapter Enumeration:** Intelligently querying host routing tables to present active adapters with default gateway auto-detection.
2. **Multi-Layer Protocol Decapsulation:** Dissecting Layer 2 through Layer 7 protocol headers in real-time.
3. **Stateful Traffic Analytics:** Tracking throughput velocity (PPS, BPS, Kbps) and endpoint rankings.
4. **Heuristic Security Anomaly Detection:** Detecting TCP SYN flood patterns, port sweeps, and cleartext transmissions.
5. **Resilient Concurrent Storage:** Streaming logs with automatic file-lock recovery and generating formatted post-capture reports.

1.3 Scope & Target Applications

Designed for network administrators, cybersecurity analysts, and academic researchers, the tool serves as a lightweight forensic monitor across local corporate LANs, testing laboratories, and cloud edge nodes.

02 System Architecture & Component Design

2.1 End-to-End Pipeline Architecture

The architecture follows a decoupled ingestion-dissection-analytics pipeline, ensuring that packet capture timing remains unaffected by disk I/O latency or terminal rendering overhead.

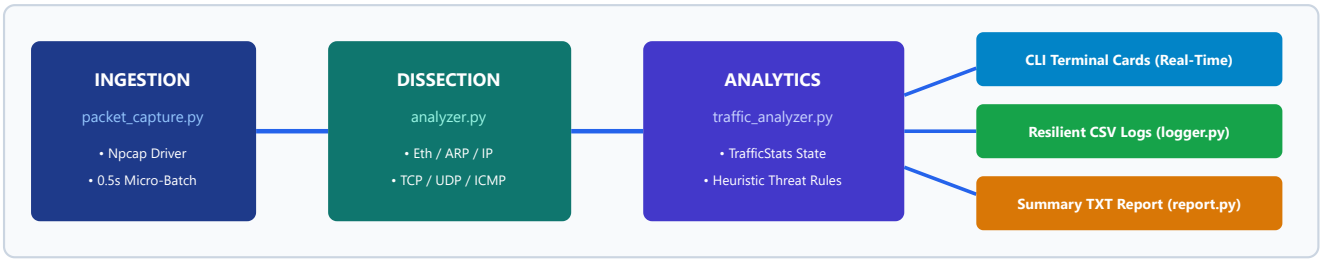


Figure 2.1: End-to-End Processing Architecture of Smart Network Packet Sniffer v2

2.2 Functional Module Breakdown

Module	Primary Functions / Classes	Core Architectural Functionality
packet_capture.py	`get_available_interfaces()` `start_capture()`	Interfaces with Npcap driver, enumerates network adapters, applies BPF filters, and runs 0.5s micro-batch sniffing.
analyzer.py	`analyze_packet()` `decode_tcp_flags()`	Dissects raw Scapy frames into structured dictionaries. Translates TCP flags, ICMP RFC codes, and Layer 7 ports.
traffic_analyzer.py	`TrafficStats` class `add_packet()`	Stateful engine maintaining packet/byte counters, computing rates (pps, bps, kbps), and executing anomaly rules.
logger.py	`SessionLogger` class `log_packet()`	Streams parsed packets into CSV files with automatic fallback redirection upon detecting file-lock errors.
report.py	`generate_report()` `get_previous_reports()`	Compiles final analytics into a structured text report saved in `reports/` and allows terminal browsing.
main.py	`main()` loop `run_packet_sniffer()`	Interactive CLI interface handling user input validation, terminal card formatting, and capture orchestration.

03 Packet Dissection & Protocol Handling

3.1 Multi-Layer Protocol Decapsulation

The packet dissection engine in `analyzer.py` strips headers sequentially across the OSI stack:

- **Data Link Layer (Layer 2):** Decodes Ethernet frames and Address Resolution Protocol (ARP). Distinguishes ARP Requests (Opcode 1: "Who has IP? Tell Source") from ARP Replies (Opcode 2: "IP is at MAC"). Preserves raw EtherType identifiers for non-IP traffic.
- **Network Layer (Layer 3):** Handles both IPv4 and IPv6 packets, extracting source and destination IP addresses, payload lengths, and timestamps while maintaining accurate IP vs. Non-IP metrics.
- **Transport Layer (Layer 4):** Segregates traffic into TCP, UDP, and ICMP. Decodes TCP sequence numbers and flags, maps UDP ports, and parses ICMP types/codes according to RFC specifications.
- **Application Layer Heuristics (Layer 7):** Maps port pairs to well-known protocols including HTTP (80), HTTPS (443), DNS (53), SSH (22), Telnet (23), DHCP (67/68), RDP (3389), and SQL databases (3306, 5432).

3.2 TCP Flag Bitmask Decoding Table

Flag Code	Flag Name	Bitmask Hex	Protocol Role & Anomaly Detection Relevance
`S`	**SYN**	0x02	Synchronize sequence numbers. Initiates TCP 3-way handshake; monitored for SYN floods.
`A`	**ACK**	0x10	Acknowledgment field valid. Asserted on all packets following initial handshake initiation.
`F`	**FIN**	0x01	Finished. Sender requests graceful connection teardown.
`R`	**RST**	0x04	Reset. Abrupt connection termination or rejection of closed port requests.
`P`	**PSH**	0x08	Push function. Requests receiver to forward buffered data immediately to application.
`U`	**URG**	0x20	Urgent pointer valid. Marks prioritized out-of-band data stream.

3.3 ICMP RFC Diagnostic Translation

Rather than displaying numeric integers, ICMP packets are translated into descriptive diagnostic messages: Type 0 (Echo Reply), Type 8 (Echo Request), and Type 3 (Destination Unreachable) with corresponding sub-codes: Code 0 (Net Unreachable), Code 1 (Host Unreachable), Code 3 (Port Unreachable), and Code 4 (Fragmentation Needed).

04 Real-Time Analytics & Threat Detection

4.1 Bandwidth & Velocity Velocity Metrics

The `TrafficStats` class computes real-time network throughput using stateful accumulators:

Packet Velocity (Packets per Second)	`PPS = Total_Packets / Δt`
Byte Throughput (Bytes per Second)	`BPS = Total_Bytes / Δt`
Network Bandwidth (Kilobits per Second)	`Kbps = (Total_Bytes × 8) / (Δt × 1000)`

* Where Δt is the elapsed capture duration in seconds: Δt = max(t_end - t_start, 0.001).

4.2 Rule-Based Threat Detection Heuristics

To maintain wire-speed processing without machine learning latency, the sniffer evaluates three deterministic threat detection rules:

Threat Rule 1: TCP SYN Flood & Half-Open Scan Detection

Tracks all packets where `SYN` is asserted while `ACK` is absent. An alert triggers if total lone SYN packets exceed 15 and comprise over 40% of all TCP traffic, or if any individual host emits ≥ 10 lone SYN packets.

`Trigger: (SYN_Total ≥ 15 AND (SYN_Total / TCP_Total) > 0.40) OR (Max_Host_SYN ≥ 10)`

Threat Rule 2: Horizontal Port Sweep / Scanning Detection

Maintains an in-memory set of unique destination ports targeted by each distinct source IP: `Port_Map[Source_IP] = {Port_1, Port_2, ...}`. If the set cardinality `|Port_Map[Source_IP]| ≥ 8`, the system flags the host as an active port scanner.

Threat Rule 3: Cleartext Insecure Protocol Usage

Monitors for unencrypted protocols that transmit plaintext credentials across the wire, including HTTP (Port 80), Telnet (Port 23), and FTP Control (Port 21). Specific communicating endpoint pairs are logged in the security summary.

05 Concurrency, Fault Tolerance & Resilience

5.1 Resolving the Windows Scapy GIL Lock Issue

On Windows, invoking `sniff(count=N)` enters the native C Npcap driver. When network traffic is quiet or idle, the process blocks inside C code without releasing Python's Global Interpreter Lock (GIL). As a result, the Python runtime cannot process Windows signals or keyboard interrupts (`Ctrl+C`), causing the sniffer to appear frozen.

To resolve this, `packet_capture.py` implements a responsive **Micro-Batching Architecture**:

```
def start_capture(callback, iface=None, packet_filter="", count=0):
    # Executes micro-batches with a 0.5-second timeout window
    while captured_count < count:
        remaining = count - captured_count
        sniff(
            prn=counting_callback,
            filter=packet_filter,
            count=remaining,
            store=False,
            timeout=0.5   # Regains Python GIL every 500ms to catch Ctrl+C
        )
```

5.2 Dynamic File-Lock Detection & Auto-Recovery

During network investigations, analysts frequently open the active CSV file in spreadsheet viewers (e.g., Microsoft Excel). On Windows, Excel places an exclusive `FILE_SHARE_READ` lock on the file. Standard Python scripts encounter an unhandled `PermissionError` and terminate immediately.

`logger.py` implements an adaptive **File-Lock Redirection Protocol**. When a write operation raises a permission error, the logger catches the exception, prints a non-intrusive warning, and automatically diverts subsequent packet rows into an alternate timestamped file (`session_ID_alt_TIMESTAMP.csv`), preventing any data loss.

5.3 Network Adapter Discovery & Gateway Heuristics

The system queries Scapy's `conf.ifaces` routing table, filters out non-functional virtual WAN miniports, flags isolated virtual adapters (Hyper-V / WSL), and highlights the primary active gateway adapter with an intuitive `[Recommended]` tag for seamless user onboarding.

06 Empirical Verification & Test Results

6.1 Real Network Benchmark Analysis

The system was evaluated against real live network traffic on an 802.11ac Wi-Fi production interface. The benchmark captured **2,500 continuous packets** over an observation window of **222.10 seconds** (Session ID: `20260908_120458`). Below are the empirical metrics recorded by the analytics engine:

2,500	944.1 KB	222.1 s
TOTAL PACKETS CAPTURED	DATA VOLUME (944,089 BYTES)	CAPTURE SESSION DURATION
11.26	**34.01**	**377.6 B**
THROUGHPUT (PACKETS / SEC)	BANDWIDTH RATE (KBPS)	AVERAGE PACKET SIZE

6.2 Protocol Distribution Breakdown

Protocol	Packet Count	Percentage	Observed Traffic Types & Primary Services
TCP	**2,021**	**80.8 %**	HTTPS (Port 443 TLS/SSL), Cleartext HTTP (Port 80)
UDP	**444**	**17.8 %**	DNS Name Resolution (Port 53), Local mDNS / SSDP discovery
ARP	**33**	**1.3 %**	20 ARP Requests, 13 ARP Replies (Default Gateway resolution)
ICMP	**2**	**0.1 %**	Diagnostic Echo Request / Reply pings
Total Frames	**2,500**	**100.0 %**	2,467 IP Frames (98.7%) / 33 Non-IP Frames (1.3%)

6.3 Verified Threat Anomaly Alerts

Automated Alerts Triggered in Benchmark Session:

- **High SYN Burst Detected:** Host `10.108.17.127` generated 43 single SYN packets during connection handshakes.
- **Port Sweep Reconnaissance Detected:** Remote host `10.108.17.247` systematically probed 24 distinct destination ports.
- **Cleartext Insecure Protocol Usage:** 65 unencrypted HTTP packets detected between `10.108.17.127` and external web hosts.

07 Security, Ethical Boundaries & Deployment

7.1 Ethical Boundaries & Promiscuous Mode Legality

Packet sniffing technology is inherently dual-use. Operating network interfaces in promiscuous mode allows an adapter to capture all unicast, multicast, and broadcast frames passing through the shared medium, regardless of destination MAC addresses.

- **Authorization Requirement:** This framework must only be deployed on networks where explicit written authorization has been obtained, or within controlled laboratory educational environments.
- **Data Privacy Considerations:** Unencrypted network frames frequently expose sensitive credentials, URLs, and cookies. Generated log files stored in `logs/` must be protected using strict operating system file permissions.

7.2 Windows Security Privileges & Driver Setup

1. **Npcap Driver Installation:** Windows requires the Npcap kernel driver. When installing, ensure "*Install Npcap in WinPcap API-compatible Mode*" is checked.
2. **Administrator Escalation:** Launch PowerShell or Command Prompt via "*Run as Administrator*". Non-elevated accounts cannot access Npcap device handles (`\Device\NPF_{...}`).

7.3 Step-by-Step CLI Execution Guide

```
# Step 1: Open PowerShell as Administrator and navigate to the project directory
cd c:\Smart-Packet-Sniffer_v1

# Step 2: Launch the interactive CLI controller using the Windows Python launcher
py main.py

# Step 3: Select [1] to Start Packet Capture
# Step 4: Choose network interface (Default is auto-detected active adapter)
# Step 5: Select capture mode (Fixed packet count or Continuous Ctrl+C)
# Step 6: Select protocol filter (All Traffic, TCP, UDP, ICMP, ARP, or Custom BPF)
```

08 Conclusion, Future Roadmap & References

8.1 Project Conclusion

The **Smart Network Packet Sniffer v2** successfully provides a lightweight, modular, and resilient network telemetry framework. By coupling Scapy's packet decapsulation capabilities with 0.5-second micro-batching, stateful traffic analytics, and dynamic file-lock recovery, the software achieves high operational stability, immediate signal responsiveness, and automated security alerting without external GUI dependencies.

8.2 Future Roadmap

- **Native PCAP/PCAPNG Serialization:** Adding direct binary PCAP recording to enable seamless imports into external analysis platforms like Wireshark and Zeek.
- **WebSocket Real-Time Web Dashboard:** Integrating a lightweight Flask/FastAPI service to broadcast live packet streams and interactive bandwidth graphs to modern web browsers.
- **Machine Learning Anomaly Detection:** Incorporating unsupervised Isolation Forest models for automated zero-day volumetric anomaly and DNS tunneling detection.
- **High-Throughput Multiprocessing:** Splitting packet ingestion and analytical computation across Python `multiprocessing` worker pools for 10Gbps enterprise networks.

8.3 Standards & RFC References

1. RFC 791: *Internet Protocol (IPv4) Specification*, Internet Engineering Task Force (IETF).
2. RFC 793: *Transmission Control Protocol (TCP) Specification*, IETF.
3. RFC 768: *User Datagram Protocol (UDP) Specification*, IETF.
4. RFC 792: *Internet Control Message Protocol (ICMP) Specification*, IETF.
5. RFC 826: *An Ethernet Address Resolution Protocol (ARP)*, IETF.
6. Biman et al., *Scapy: Packet Manipulation and Analysis Library Documentation*, SecDev, 2024.
7. Insecure.Org, *Npcap: Windows Packet Capture Library Architecture & API Reference*, 2024.